

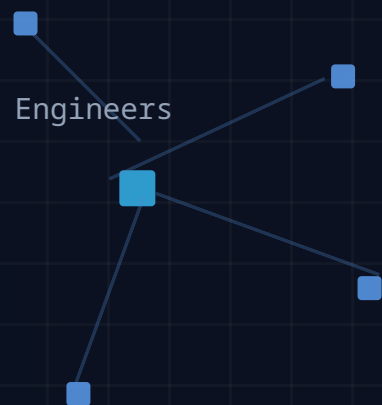
ALGOTUTOR

TOP 20 KUBERNETES INTERVIEW QUESTIONS

Practical Questions • Expert Answers • Interview
Insights

// A Guide for DevOps, Cloud & Software Engineers

2026 INTERVIEW EDITION



DESIGNED FOR LEARNERS

BUILD SKILLS THAT MATTER

WHY THIS GUIDE?

Kubernetes has become the de facto standard for container orchestration. Whether you are aiming for a DevOps, Cloud Engineering, or Backend role, Kubernetes knowledge is no longer optional—it is heavily tested in modern technical interviews.

This guide moves beyond generic definitions. It is curated for real-world interviews, focusing on concepts that engineering managers actually care about: architecture, workload management, networking, and troubleshooting.

WHAT YOU'LL LEARN

- **Kubernetes Architecture:** Control Plane, Kubelet, etcd.
- **Core Workloads:** Pods, Deployments, ReplicaSets, StatefulSets.
- **Networking & Services:** ClusterIP, NodePort, LoadBalancer, Ingress.
- **Configuration:** ConfigMaps, Secrets, RBAC.
- **Operations:** Probes, Requests/Limits, Self-healing, Troubleshooting.

AlgoTutor Learning Note

"Don't just memorize Kubernetes commands. Understand the distributed system behind them. Interviewers look for engineers who understand *desired state vs. actual state*."

LEVEL 1 — KUBERNETES FUNDAMENTALS

QUESTION #1

What is Kubernetes?

● Beginner

Short Answer:

Kubernetes (K8s) is an open-source container orchestration platform designed to automate the deployment, scaling, and management of containerized applications. It acts as a control plane that ensures the actual state of the cluster matches the desired state defined by the user.

Detailed Explanation:

While container runtimes (like containerd or Docker) run individual containers on a single host, Kubernetes groups these containers into logical units (Pods) and distributes them across a cluster of machines. It abstracts away the underlying infrastructure, providing built-in mechanisms for load balancing, auto-scaling, rollouts, and self-healing. When a node fails, Kubernetes automatically reschedules the affected containers to healthy nodes without manual intervention.

Key Points:

- Eliminates single points of failure through automated rescheduling.
- Operates on a declarative model (YAML/JSON manifests).
- Agnostic to underlying infrastructure (runs on AWS, GCP, Azure, or On-Prem).
- Does NOT run containers directly; it orchestrates a container runtime (e.g., containerd, CRI-O).

Interview Tip

Always mention the "Declarative Model" and "Desired State vs Actual State." This shows you understand the fundamental philosophy of Kubernetes.

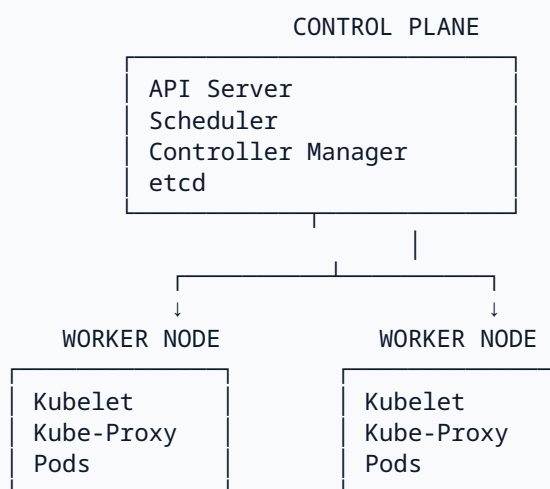
QUESTION #2

What are the main components of Kubernetes architecture?

Beginner

Short Answer:

A Kubernetes cluster consists of a Control Plane (master) and Worker Nodes. The Control Plane makes global decisions about the cluster, while Worker Nodes host the actual application workloads (Pods).



Detailed Explanation:

- **API Server:** The front-end of the control plane; all commands (`kubectl`) pass through here.
- **etcd:** A highly available key-value store containing the cluster's state and configuration.
- **Scheduler:** Watches for newly created Pods and assigns them to nodes based on resource availability.
- **Controller Manager:** Runs controller processes (like Node Controller, ReplicaSet Controller) to reconcile the actual state with the desired state.
- **Kubelet (Worker):** An agent running on each node that ensures containers are running in a Pod.
- **Kube-proxy (Worker):** Maintains network rules on nodes to allow communication to Pods.

Interviewer Insight

Interviewers frequently ask "Where is the cluster state stored?" The answer is always

etcd

. If etcd is lost, the cluster is lost.

QUESTION #3

What is a Pod?

● Beginner

Short Answer:

A Pod is the smallest, most basic deployable object in Kubernetes. It represents a single instance of a running process in your cluster and can contain one or multiple closely coupled containers.

Detailed Explanation:

Kubernetes does not deploy containers directly; it wraps them in a Pod. All containers within the same Pod share the same local network space (IP address and port space) and storage volumes. They communicate with each other using `localhost`. While multi-container Pods exist (often using a "sidecar" pattern for logging or proxies), the most common pattern is one container per Pod.

Key Points:

- Smallest unit of execution in Kubernetes.
- Containers in a Pod share an IP address and Network Namespace.
- Pods are ephemeral—they are meant to be destroyed and recreated, not patched.

Common Mistake

Never say "Kubernetes schedules containers." Kubernetes schedules
Pods

.

QUESTION #4

Pod vs Container

● Beginner

Short Answer:

A container is an isolated execution environment for an application and its dependencies. A Pod is a Kubernetes abstraction that wraps one or more containers to manage them as a single deployable unit.

Aspect	Container	Pod
Scope	Application code and dependencies.	Kubernetes deployment unit wrapping containers.
Networking	Isolated unless bound to a port.	All containers inside share the same IP.
Lifecycle	Managed by CRI (containerd, Docker).	Managed by Kubernetes (Kubelet).

QUESTION #5

What is a Deployment?

● Beginner

Short Answer:

A Deployment is a higher-level Kubernetes controller that manages the declarative updates, scaling, and rollbacks of ReplicaSets and Pods.

Detailed Explanation:

You rarely create Pods directly because they are ephemeral. Instead, you define a Deployment, which guarantees that a specific number of Pod replicas are running at any given time. Deployments allow you to rollout updates (e.g., updating an image from v1 to v2) seamlessly using strategies like Rolling Updates, ensuring zero downtime.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    # Pod Template goes here
```

Interview Tip

Highlight that Deployments manage stateless applications and provide built-in versioning and rollback capabilities (`kubectl rollout undo deployment/x`).

LEVEL 2 — CORE KUBERNETES OBJECTS

QUESTION #6

What is a ReplicaSet?

● Beginner

Short Answer:

A ReplicaSet ensures that a specified number of Pod replicas are running at any given time. If a node goes down and a Pod is lost, the ReplicaSet controller notices the deficit and creates a new Pod to maintain the desired count.

Detailed Explanation:

While you can create a ReplicaSet directly, it is highly recommended to use Deployments instead. A Deployment automatically manages underlying ReplicaSets. When you update a Deployment's image, it creates a *new* ReplicaSet, scales it up, and scales the *old* ReplicaSet down.

Common Mistake

Candidates often confuse Deployments and ReplicaSets. Deployment handles the **rollout/upgrade** strategy. ReplicaSet purely handles **maintaining the replica count**.

QUESTION #7

What is a Kubernetes Service?

● Beginner

Short Answer:

A Service is an abstraction that provides a stable IP address and DNS name for a logical set of Pods, acting as an internal load balancer.

Detailed Explanation:

Pods are ephemeral; they are constantly created and destroyed, and their IP addresses change every time. If a frontend application needs to talk to a backend, it cannot rely on Pod IP addresses. A Service provides a static, virtual IP. It uses label selectors to identify the target Pods and routes traffic to them, regardless of their individual IP addresses.

QUESTION #8

ClusterIP vs NodePort vs LoadBalancer

Intermediate

Short Answer:

These are the three primary Service types in Kubernetes, determining how the service is exposed to the network.

Type	Accessibility	Use Case
ClusterIP	Internal cluster access only. (Default)	Backend databases or internal microservices.
NodePort	Exposes Service on a static port on every Node's IP.	Direct external access for testing or simple routing.
LoadBalancer	Provisions an external cloud Load Balancer (AWS/GCP).	Production web apps needing a public entry point.

Interviewer Insight

Interviewers want to see that you understand the progression: LoadBalancer sits on top of NodePort, which sits on top of ClusterIP. They are layered.

QUESTION #9

What is a Namespace?

Intermediate

Short Answer:

A Namespace is a logical partition within a single Kubernetes cluster. It allows multiple teams or environments (e.g., dev, staging, prod) to share the same physical cluster without stepping on each other's toes.

Key Points:

- Names of resources must be unique within a namespace, but not across namespaces.
- Resource Quotas can be applied per namespace to prevent resource hogging.
- Default namespaces include: `default`, `kube-system`, and `kube-public`.

QUESTION #10

ConfigMaps vs Secrets

Intermediate

Short Answer:

Both separate configuration from application code, allowing you to mount data into Pods as environment variables or files. ConfigMaps store non-confidential data, while Secrets store sensitive data (like passwords or API keys).

Detailed Explanation:

By default, Kubernetes Secrets are merely Base64 encoded, **not encrypted**. They provide a structural separation so you can manage RBAC differently (e.g., preventing a developer from reading Secrets while allowing ConfigMap access).

Common Mistake

Assuming Secrets are highly secure by default. In production, you must enable "Encryption at Rest" for etcd, or use external secret managers like HashiCorp Vault or AWS Secrets Manager.

LEVEL 3 — APPLICATION MANAGEMENT

QUESTION #11

Liveness vs Readiness vs Startup Probes

Intermediate

Short Answer:

Probes are diagnostic checks performed by the Kubelet to monitor container health.

- **Liveness Probe:** Checks if the container is running. If it fails, Kubelet kills and restarts the container. (Prevents deadlock).
- **Readiness Probe:** Checks if the application is ready to receive traffic. If it fails, the Pod's IP is removed from Service endpoints.
- **Startup Probe:** Used for slow-starting legacy apps. Disables liveness/readiness checks until the app successfully starts.

Interviewer Insight

Interviewers asking about readiness probes are usually checking whether you understand the difference between a process running (Liveness) and an application being actually ready to process requests (Readiness).

QUESTION #12

Resource Requests vs Limits

Intermediate

Short Answer:

Requests define the guaranteed minimum CPU and memory a container needs. Limits define the absolute maximum CPU and memory a container can consume.

Detailed Explanation:

The Kubernetes Scheduler uses **Requests** to decide which node to place the Pod on. If a node doesn't have enough available requested resources, the Pod stays Pending. **Limits** are enforced by the container runtime at the OS level (cgroups). If a container exceeds its Memory Limit, it is killed with an OOMKilled error. If it hits its CPU Limit, it is throttled but not killed.

QUESTION #13

What is a StatefulSet?

Intermediate

Short Answer:

A StatefulSet is the workload API object used to manage stateful applications (like databases). Unlike Deployments, it maintains a sticky identity for each of its Pods.

Detailed Explanation:

In a Deployment, Pods are interchangeable (`pod-x6f9`, `pod-a2b1`). In a StatefulSet, Pods get stable, sequential hostnames (`db-0`, `db-1`, `db-2`). They also get stable storage via PersistentVolumeClaims (PVCs). If `db-1` is deleted, it is recreated as `db-1` and reattaches to the exact same storage volume.

QUESTION #14

What is a DaemonSet?

Intermediate

Short Answer:

A DaemonSet ensures that a copy of a specific Pod runs on ALL (or a selected subset of) nodes in the cluster.

Practical Example:

DaemonSets are almost exclusively used for cluster-wide infrastructure agents, such as:

- Log collectors (Fluentd, Promtail)
- Node monitoring agents (Prometheus Node Exporter, Datadog)
- Network plugins (Calico, Flannel)

QUESTION #15

Job vs CronJob

Intermediate

Short Answer:

A **Job** creates one or more Pods and ensures that a specified number of them successfully terminate (run to completion). A **CronJob** is a Job that runs on a time-based schedule.

Key Points:

- Unlike Deployments (which restart Pods forever), Jobs are for batch tasks (e.g., database migrations, data processing).
- CronJobs use standard cron format (e.g., `0 0 * * *` for daily at midnight).

LEVEL 4 — NETWORKING, SECURITY & OPERATIONS

QUESTION #16

What is Ingress?

● Advanced

Short Answer:

Ingress is an API object that manages external access to the services in a cluster, typically HTTP/HTTPS. It provides routing rules, SSL termination, and name-based virtual hosting.

Detailed Explanation:

Instead of creating a cloud LoadBalancer for every single microservice (which is expensive), you create one Ingress Controller (like Nginx Ingress). The Ingress Controller gets the single public IP. You then write Ingress resources (YAML) that tell the controller: "Route `api.algotutor.com/users` to the User Service, and `api.algotutor.com/courses` to the Course Service."

Common Mistake

Confusing an Ingress resource with an Ingress Controller. The resource is just a set of routing rules (YAML). The Controller is the actual running proxy (e.g., NGINX, Traefik) that enforces those rules.

QUESTION #17

How does Kubernetes perform self-healing?

● Advanced

Short Answer:

Kubernetes self-heals by running continuous control loops that check the actual state against the desired state defined in etcd. If a discrepancy is found, Kubernetes acts to correct it.

Examples:

- **Pod Crash:** Kubelet restarts containers that crash based on restart policies and Liveness Probes.
- **Node Failure:** If a Worker Node dies, the Controller Manager sees the missed heartbeats. The ReplicaSet controller notices Pods are missing and schedules replacements on healthy nodes.
- **Unresponsive App:** Readiness probes detect failure and remove the Pod from Service load-balancing, stopping traffic to the broken app.

QUESTION #18

How do you troubleshoot a failing Kubernetes Pod?

● Advanced

Short Answer:

Troubleshooting follows a structured path of checking status, events, logs, and interactive debugging.

Detailed Steps:

1. `kubectl get pods` (Look at the STATE: CrashLoopBackOff, Pending, ImagePullBackOff?)
2. `kubectl describe pod <pod-name>` (Scroll to the bottom "Events" section to see scheduling errors, probe failures, or image pull errors).
3. `kubectl logs <pod-name>` (Check application-level stdout/stderr logs. Use `-p` to see logs from a previously crashed container).
4. `kubectl exec -it <pod-name> -- sh` (Shell into the container to test network connectivity or check files).

QUESTION #19

What are RBAC and ServiceAccounts?

● Advanced

Short Answer:

RBAC (Role-Based Access Control) is the system for regulating access to Kubernetes API resources. A ServiceAccount is an identity given to a Pod so it can securely communicate with the Kubernetes API.

Detailed Explanation:

Users (humans) are authenticated externally (OIDC, AWS IAM). ServiceAccounts are for machine-to-machine authentication within the cluster.

RBAC uses three concepts:

1. **Role / ClusterRole:** Defines **what** can be done (e.g., "get pods", "delete deployments").
2. **Subject:** Defines **who** (User, Group, or ServiceAccount).
3. **RoleBinding / ClusterRoleBinding:** Connects the **who** to the **what**.

QUESTION #20

What happens when you run `kubectl apply -f deployment.yaml`?

● Advanced

Short Answer:

This tests your end-to-end understanding of the architecture. The command triggers a chain reaction from the API server down to the container runtime.

The Flow:

1. `kubectl` validates the YAML and sends an HTTP POST/PUT to the **API Server**.
2. API Server authenticates, authorizes, and persists the desired state in **etcd**.
3. The **Controller Manager** (specifically Deployment controller) sees the new state, and creates a ReplicaSet. The ReplicaSet controller sees the requirement and creates Pod objects in etcd (Status: Pending).
4. The **Scheduler** notices unassigned Pods and assigns them to an optimal Worker Node by updating the Pod definition with a `nodeName`.
5. The **Kubelet** on the chosen Node sees the assignment, instructs the Container Runtime (e.g., containerd) to pull the image and start the container, and reports the status back to the API Server.

KUBERNETES INTERVIEW QUICK REVISION

1. Architecture Flow

API Server → etcd → Scheduler → Controllers → Kubelet → Runtime

2. Core Objects

Object	Purpose
Pod	Smallest unit, wraps containers, shares network/IP.
Deployment	Stateless apps, manages ReplicaSets, rollouts, scaling.
ReplicaSet	Maintains exact pod count.
Service	Stable IP/DNS for Pods, internal load balancer.

3. Workloads

Type	Best For
Deployment	Web servers, APIs (Stateless).
StatefulSet	Databases, Message Queues (Stateful, stable network/storage ID).
DaemonSet	Logging agents, Monitoring agents (1 per node).
Job / CronJob	Batch processing, DB migrations, scheduled tasks.

4. Configuration & Storage

ConfigMap: Plain text config data.

Secret: Base64 encoded sensitive data.

PVC (PersistentVolumeClaim): Pod's request for storage.

5. Networking

ClusterIP: Internal only.

NodePort: Exposes on node IP port (30000-32767).

LoadBalancer: Cloud LB provisioning.

Ingress: HTTP/S routing rules, URL paths, SSL.

6. Troubleshooting CLI

1. `kubectl get pods`
2. `kubectl describe pod <name>`
3. `kubectl logs <name>`
4. `kubectl get events --sort-by='.metadata.creationTimestamp'`

KEEP LEARNING. KEEP BUILDING.

Your next interview starts with what you learn today.

At AlgoTutor, we believe continuous learning and practical engineering are the keys to career growth. Building projects, mastering system design, and understanding the core architecture behind tools like Kubernetes will set you apart in any technical interview.

Follow AlgoTutor for:

- Technical Roadmaps
- Interview Guides
- Cheat Sheets
- Career Resources
- Engineering Insights

AlgoTutor

Learn. Build. Get Interview Ready.

LinkedIn: [linkedin.com/company/algotutor](https://www.linkedin.com/company/algotutor)

Website: www.algotutor.io